

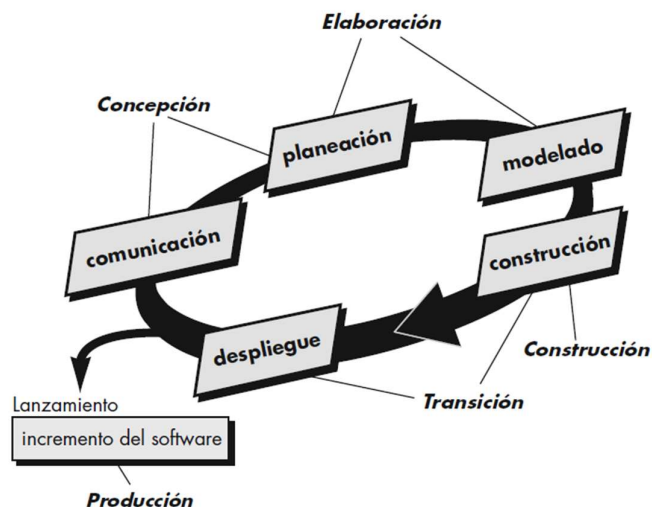
Contenido

PROCESO UNIFICADO DE DESARROLLO (PUD).	2
CONCEPTOS BASICOS.	3
FLUJO DE TRABAJO → Conjunto de actividades referidas al desarrollo de un sistema, referidas a un aspecto de este en particular (requisitos, análisis, diseños) estableciendo los trabajadores que participan, las actividades que se realizan y los artefactos que se producen.	3
TRABAJADOR → Rol de personas responsables de realizar las actividades y producir los artefactos.	3
ACTIVIDAD → Las distintas tareas a realizar en cada flujo de trabajo para realizar los artefactos necesarios.	3
ARTEFACTO → Resultado que se produce o elabora en un flujo de trabajo.	3
ASPECTOS CARACTERISTICOS DEL PUD.	3
EL CICLO DE VIDA DEL PUD.	4
FASES DEL CICLO DE VIDA DEL PUD.	4
1. Fase de inicio o concepción	4
2. Fase de elaboración	4
3. Fase de construcción:	4
4. Fase de transición	4
Modelo de Análisis. Resumen.	5
¿Cuándo ocurre que los objetos se comportan de manera distinta conforme al paso del tiempo?	5
¿Entonces cuándo se necesita un diagrama de máquina de estado?	5
Cuadro conceptual comparativo del modelo de requerimientos y de análisis.	6
Cuadro conceptual comparativo del modelo de CU y de análisis.	6
Describe detalladamente como está compuesto el modelo resultante del workflow de análisis.	7
Modelo de análisis:	7
Clases de análisis:	7
Realizaciones de CU de análisis:	7
Paquetes del análisis:	8
Descripción de la arquitectura (Vista del modelo de análisis):	8
Explique en qué casos recomienda el Proceso Unificado de Desarrollo la realización de un Modelo de Análisis, y que rol tiene el Análisis en el contexto del ciclo de vida iterativo e incremental, que propone el PUD.	8
Explique los tres tipos de clases que se utilizan para modelar el análisis, según el PUD y ejemplifíquelas.	9
Clases de análisis	9
Describe las responsabilidades de los trabajadores involucrados en el Flujo de Trabajo de Análisis y vincúlelos con los artefactos que producen.	9
Explique cuáles son los artefactos de salida del Workflow de Análisis y describa los diagramas de UML 2.0 que se utilizan para construirlos.	10
Explique por qué es necesario considerar el análisis como una etapa diferente a la captura de requisitos o del diseño y la implementación.	11
UML 2.0:	12

¿Cuáles son los diagramas que UML 2.0? propone para crear modelos durante el Workflow de Análisis ? Explique el propósito de cada uno, sus características y los conceptos que modela.	12
Mencione y describa los diagramas de UML 2.0 que se clasifican como diagramas de estructura .	12
Especifique como está compuesto el modelo conceptual de UML 2.0, y describa en términos generales los elementos que lo componen.	13
a) Elementos.	13
b) Relaciones	13
c) Diagramas.	13
¿Qué modelan los diagramas de interacción ? Realice una clasificación de cada uno de ellos.	14
Explique la diferencia entre los conceptos: Modelo, Vista y Diagrama. Ejemplifique.	14
¿Cuáles son los diagramas de UML 2.0 que modelan objetos (instancias de clases)? Describa cada uno de ellos y mencione en que workflows del Proceso Unificado se los puede utilizar.	15
¿Cuáles son los diagramas que UML 2.0 propone para modelar la estructura del software? Considere los que podemos aplicar en el Flujo de Trabajo de análisis: explique el propósito de cada uno y en qué actividad del F.T. se utilizan.	15
Patrones GRASP:	16
Mencione los patrones GRASP utilizados en el Análisis, describa cuál es su utilidad y desarrolle un ejemplo PROPIO en el que aplique los patrones CREADOR y BAJO ACOPLAMIENTO.	16
¿Para qué se aplican los Patrones GRASP? ¿En qué actividad del F.T. de Análisis se hace uso de los mismos?	17
Completar el siguiente cuadro para que el patrón se corresponda con la afirmación:	17
Para el patrón Experto plantee un ejemplo PROPIO concreto de:	17
¿Cuál es la utilidad de los patrones GRASP? Explique su propósito y desarrolle un ejemplo PROPIO en el que use el patrón CONTROLADOR y un ejemplo en el que no lo use.	18
Resumen de clasificación de PATRONES GRASP.	18

PROCESO UNIFICADO DE DESARROLLO (PUD).

El Proceso Unificado de Desarrollo (PUD) es un **marco de trabajo** ampliamente utilizado en el desarrollo de software que *integra las mejores prácticas de diversos métodos y enfoques*. El proceso unificado reconoce la importancia de la **comunicación con el cliente** y los **métodos directos** para describir su punto de vista respecto de un sistema (el **caso de uso**). Hace **énfasis** en la importancia de la **arquitectura del software**. Sugiere un **flujo del proceso iterativo e incremental**, lo que da la sensación evolutiva.



Al principio de la década de 1990, James Rumbaugh, Grady Booch e Ivar Jacobson **crearon un método unificado que combina lo mejor de cada método individual de análisis y diseño**

orientado a objetos. Como resultado se obtiene UML, *lenguaje de modelado unificado*, que contiene una notación robusta para el modelado y desarrollo de los sistemas orientados a objetos.

En síntesis:

- ✓ Utiliza el **Lenguaje Unificado de Modelado (UML)** para preparar todos los esquemas de un sistema de software.

UML: *Lenguaje estándar para escribir planos de software, que permite visualizar, especificar, construir y documentar artefactos de un sistema.*

- ✓ **PUD** establece actividades para crear distintos modelos, utilizando UML para su representación, considerando la filosofía del paradigma orientado a objetos. El UML brinda la tecnología necesaria para apoyar la práctica de la ingeniería de software orientada a objetos, pero no da la estructura del proceso que guíe al equipo del proyecto cuando aplican la tecnología.

CONCEPTOS BASICOS.

FLUJO DE TRABAJO → *Conjunto de actividades referidas al desarrollo de un sistema, referidas a un aspecto de este en particular (requisitos, análisis, diseños) estableciendo los trabajadores que participan, las actividades que se realizan y los artefactos que se producen.*

TRABAJADOR → *Rol de personas responsables de realizar las actividades y producir los artefactos.*

ACTIVIDAD → *Las distintas tareas a realizar en cada flujo de trabajo para realizar los artefactos necesarios.*

ASPECTOS CARACTERISTICOS DEL PUD.

CONDUCTIDO POR CASOS DE USO.

Un caso de uso (CIJ) es un fragmento de funcionalidad del sistema que proporciona al usuario un resultado importante, **los casos de uso representan los requerimientos funcionales.**

Permiten especificar los requerimientos de un sistema.

Guían el diseño, implementación y prueba: guían el proceso de desarrollo.

Dirigido por casos de uso quiere decir que el proceso de desarrollo sigue un hilo conductor, es **decir avanza a través de una serie de flujos de trabajo que parten de la identificación de los casos de uso**, los cuales se especifican, diseñan, implementan y prueban.

CENTRADO EN LA ARQUITECTURA

Arquitectura de software: incluye los aspectos estáticos y dinámicos más significativos del sistema. Describe mediante diferentes vistas el sistema en construcción, considerando **diversos factores**, como ser la plataforma en que funcionará el software, dispositivos de hardware, conexiones de red, bloques de construcción, entre otras.

La arquitectura es una vista del diseño completo con las características más importantes resaltadas. Los casos de uso deben encajar en la arquitectura cuando se llevan a cabo. La arquitectura debe permitir el desarrollo de todos los casos de uso requeridos.

CICLO DE VIDA ITERATIVO E INCREMENTAL.

- **Iteraciones** hacen referencia a una serie de pasos en el flujo de trabajo.
- **Incrementos** refieren al crecimiento del producto.

La iteración **trata un grupo de casos de uso que juntos amplían la utilidad del producto desarrollado.**

En cada iteración se realizan actividades de análisis de requisitos, análisis propiamente dicho, diseño, implementación y prueba del sistema.

EL CICLO DE VIDA DEL PUD.

- **El proceso se repite a lo largo de una serie de ciclos que constituyen la vida de un sistema.**
- Cada ciclo produce una nueva versión del sistema para los clientes. **Cada versión es un producto preparado para su entrega.**
- *Producto:* incluye requisitos, casos de uso, especificaciones no funcionales, casos de prueba, modelo de arquitectura, código fuente, componentes ejecutables.
- **Cada ciclo consta de cuatro fases: inicio, elaboración, construcción y transición.**
- Cada fase se subdivide en iteraciones.

FASES DEL CICLO DE VIDA DEL PUD.

El ciclo se divide en **4 fases**, cada fase define **momentos de madurez del producto**, en donde al terminar una de ellas establece la **disponibilidad de un conjunto de artefactos**. Cada fase tiene mínimo **2 o más iteraciones**, siguiendo el concepto de que el **PUD es un desarrollo iterativo e incremental**.

1. Fase de inicio o concepción: Descripción del producto final, análisis del negocio, determinación de las principales funciones y arquitectura del sistema, plan del proyecto y presupuesto.

2. Fase de elaboración: Especificación de casos de uso, diseño de la arquitectura, distintas vistas arquitectónicas. El modelo de análisis desempeña un papel fundamental en la fase de elaboración del ciclo de vida del PUD, donde se enfoca en comprender en detalle los requisitos del sistema y establecer una base sólida para el diseño y la implementación. Antes de comenzar a diseñar e implementar, es necesario contar con una comprensión precisa y detallada de los requerimientos, la cual se consigue en la etapa de análisis.

3. Fase de construcción: Creación del producto, codificación. Con el objetivo de que reflejen la versión final del incremento del software.

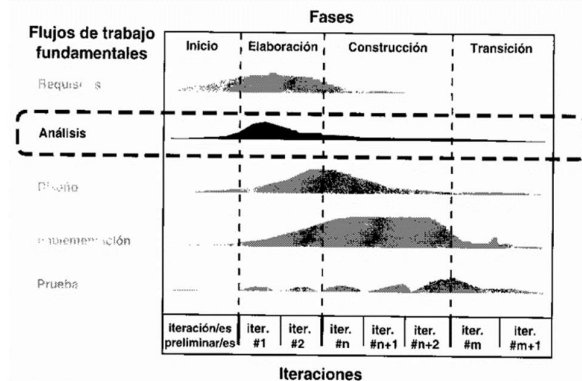
4. Fase de transición: El producto es versión beta. Instalación y prueba del sistema de parte de los usuarios, ajustes según observaciones (entrega y retroalimentación), junto con la información de apoyo (manual de usuario, de instalación, solución de problemas).

Al finalizar esta fase el software se convierte en un producto utilizable que se entrega al cliente.

WORKFLOW - [REQUERIMIENTOS].

- Desarrollar un modelo del sistema que se va a construir.
- Utiliza casos de uso para construir el modelo.

- Los requerimientos funcionales se estructura de manera natural en casos de uso.
- La mayoría de los requerimientos no funcionales pueden tratarse en el contexto de un caso de uso.



WORKFLOW - [ANÁLISIS].

- Permite una especificación más precisa de los requerimientos.
- Se describe usando el lenguaje del desarrollador.
- Estructura los requerimientos.
- Puede verse como un primer corte del modelo de diseño.

Modelo de Análisis. Resumen.

- **Ofrece una especificación más precisa de los requerimientos** que la que tenemos como resultado de la captura de requerimientos, incluyendo al modelo de casos de uso.
- Se **describe utilizando el lenguaje de los desarrolladores**, y puede por tanto introducir un mayor formalismo y ser utilizado para razonar sobre los funcionamientos internos del sistema,
- Estructura los req. de un modo que facilita su comprensión, su preparación, su modificación, y en general, su mantenimiento.

Un modelo de análisis puede considerarse como una primera aproximación al modelo de diseño (aunque es un modelo por sí mismo), y es por tanto una entrada fundamental cuando se da forma al sistema en el diseño y en la implementación. Esto se debe a que debería ser mantenible el sistema en su conjunto, y no sólo la descripción de sus requisitos.

¿Cuándo ocurre que los objetos se comportan de manera distinta conforme al paso del tiempo?

Los objetos pueden comportarse de manera diferente con el paso del tiempo **cuando necesitan cambiar su estado o su comportamiento en respuesta a eventos o condiciones específicos**. Por ello, se utiliza una máquina de estados para modelar y representar estos cambios, lo que permite una efectiva representación del comportamiento dinámico del sistema a lo largo del tiempo.

¿Entonces cuándo se necesita un diagrama de máquina de estado?

La necesidad de una máquina de estados surge **cuando los objetos en el sistema pueden estar en diferentes estados y necesitan cambiar de un estado a otro en respuesta a eventos externos o internos**. Estos eventos pueden desencadenar transiciones entre estados, y cada estado puede estar asociado con un conjunto particular de comportamientos o acciones que el objeto realiza mientras se encuentra en ese estado.

Cuadro conceptual comparativo del modelo de requerimientos y de análisis.

Aspecto	Modelo de Requerimientos	Modelo de Análisis
Perspectiva	Centrado en las necesidades y expectativas del cliente.	Centrado en la comprensión y modelado del sistema a desarrollar.
Propósito	Identificar y definir los objetivos y funcionalidades del sistema.	Descomponer y analizar las funcionalidades en detalle.
Representación	Utiliza técnicas como historias de usuario, casos de uso, etc.	Emplea diagramas de clases, 7 diagramas de comportamiento
Enfoque de abstracción	Orientado a describir qué debe hacer el sistema.	Orientado a cómo el sistema debe hacerlo y cómo se estructura.
Identificación de Actores	Identifica los usuarios y otros sistemas externos.	Analiza cómo interactúan los actores y objetos del sistema.
Funcionalidades	Describe las funcionalidades de alto nivel del sistema.	Descompone las funcionalidades en acciones más específicas.
Especificación de Requisitos	Se enfoca en los requisitos del sistema desde el punto de vista del usuario.	Detalla los requisitos funcionales y no funcionales del sistema.
Iteratividad	Puede ser iterativo para refinar y validar los requisitos.	Puede iterar para ajustar y mejorar el diseño del sistema.
Nivel de Detalle	A menudo se presenta en un nivel menos detallado y general.	Se detalla más para comprender en profundidad el sistema.

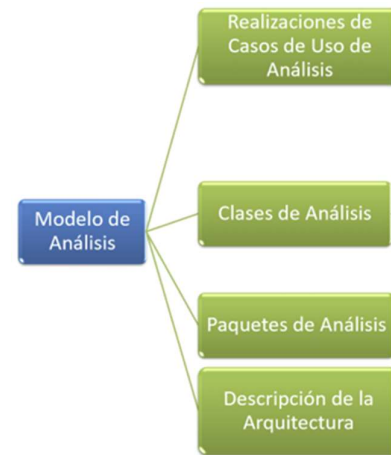
Cuadro conceptual comparativo del modelo de CU y de análisis.

Modelo de CU	Modelo de Análisis
Usa el lenguaje del cliente.	Usa el lenguaje del desarrollador.
Vista externa del sistema.	Vista interna del sistema.
Usado como contrato entre clientes y desarrolladores, para decidir que debe (y no) hacer el sistema.	Usado por los desarrolladores para saber cómo darle forma al sistema.
Estructurado por CU.	Estructurado por clases y paquetes.
Captura la funcionalidad del sistema.	Esboza como llevar a cabo la funcionalidad del sistema.
Puede contener redundancias, inconsistencias entre requerimientos.	No debe contener redundancias o inconsistencias entre requerimientos.
Define CU que serán analizados en el mod. Análisis.	Define realizaciones de CU y c/u de ellas representa el análisis de un caso de uso del modelo de casos de uso.

Describe detalladamente como está compuesto el modelo resultante del workflow de análisis.

El modelo resultante del workflow de análisis parte del:

Modelo de análisis: Modelo de objetos conceptual que nos ayuda a refinar y estructurar los requerimientos, nos permite razonar sobre **aspectos internos del sistema** y nos **proporciona una estructura centrada en el mantenimiento**. Esta puede servir como entrada para las actividades de diseño e implementación, por ello puede considerarse como *una primera aproximación al diseño*.



- El modelo hace abstracciones y evita resolver algunos problemas y tratar requerimientos que es mejor posponer al diseño e implementación (RNF).

Clases de análisis:

- Representa una abstracción de una o varias clases/subsistemas del modelo de dominio.
- Se **centra en el tratamiento de requerimientos funcionales** y pospone los no funcionales.
- Su **comportamiento se define mediante responsabilidades en un alto nivel**.
 - **Una responsabilidad es una descripción textual de un conjunto cohesivo del comportamiento de una clase.**
- Define atributos de alto nivel.
- Participa en relaciones, aunque más conceptuales que las de diseño.

Puede ser de alguno de los 3 tipos, estandarizados en UML, que ayuda a los desarrolladores a distinguir una de otra. Estas son:

1. **Clase de interfaz:** Modela las **interacciones entre un sistema y sus actores**. Estas a menudo implican presentar (y recibir) información (y peticiones) desde y hacia los usuarios y sistemas externos.
2. **Clase de entidad:** Modela el **comportamiento y la información asociada a algún fenómeno o concepto**, que posee una vida larga y a menudo es consistente.
Suelen derivarse de clases de entidad del negocio (o del dominio del problema).
3. **Clase de control:** Representa coordinación, secuencia, transacciones y control de otros objetos, y **se usa para encapsular el control de un caso de uso en concreto**.
También representa cálculos y derivaciones complejas que no pueden asociarse a ninguna clase de entidad concreta.
 - ✓ Modela los **aspectos dinámicos el sistema**.
 - ✓ Son el **nexo entre las clases de entidad y de interfaz**, ya que **no pueden relacionarse directamente**.

Realizaciones de CU de análisis: Representan una **colaboración dentro del modelo de análisis y describe como se lleva a cabo la realización de un CU en concreto**, en términos de las clases de análisis y sus objetos en **interacción, centrándose** de manera natural en los **requisitos funcionales**.

- Posee una **descripción textual del flujo de sucesos**, un diagrama de clases que muestra sus clases de análisis participantes y un diagrama de interacción que muestra la realización de un escenario particular del CU, en término de las interacciones entre objetos de dichas clases.

Paquetes del análisis: Proporcionan **un medio para organizar los artefactos del análisis en piezas manejables.**

- Pueden estar compuesto de clases de análisis, realizaciones de CU de análisis y otros paquetes.
- Pueden representar una separación de intereses de análisis, para subdividir el conocimiento del dominio y ser analizados de manera concurrente.
- Deben ser **altamente cohesivos** (*su contenido debe estar fuertemente relacionado*) y **débilmente acoplados** (*debe existir una mínima dependencia entre unos y otros*).

Descripción de la arquitectura (Vista del modelo de análisis): Contiene una **vista de la arquitectura del modelo de análisis**, que **muestra sus artefactos** más significativos de esta:

1. Clases de análisis fundamentales.
2. Realizaciones de CU que describen cierta funcionalidad crítica.
3. Descomposición del modelo de análisis en paquetes del análisis y sus dependencias.

Explique en qué casos recomienda el Proceso Unificado de Desarrollo la realización de un Modelo de Análisis, y que rol tiene el Análisis en el contexto del ciclo de vida iterativo e incremental, que propone el PUD.

Existen **tres enfoques básicos para el uso del modelo de análisis:**

1. **Mantenimiento continuo del modelo de análisis a lo largo de todo el proyecto**, obteniendo beneficios como la claridad y consistencia en cada iteración.
2. Considerar **el modelo de análisis como una herramienta transitoria**, enfocándose en él durante la fase de elaboración y dejando de actualizarlo en fases posteriores cuando el diseño y la implementación están en curso.
3. **No utilizar el modelo de análisis en absoluto**, integrando los requisitos directamente en la captura de requisitos o en el diseño, lo cual puede ser justificable en casos de requisitos simples o bien conocidos.

Al elegir entre estas variantes, es crucial realizar un análisis de coste/beneficio para determinar si es beneficioso mantener o dejar de actualizar el modelo de análisis a lo largo del proyecto. Aunque evitar el uso del modelo de análisis puede ahorrar costos, generalmente sus ventajas superan los costos asociados, por lo que se recomienda su utilización, a menos que el sistema sea excepcionalmente simple.

El modelo de análisis desempeña un papel fundamental en la fase de elaboración del ciclo de vida del PUD, donde se enfoca en comprender en detalle los requisitos del sistema y establecer una base sólida para el diseño y la implementación.

Durante la etapa de elaboración en el Proceso Unificado de Desarrollo (PUD), **las iteraciones iniciales se enfocan en el análisis**, lo que contribuye a **establecer una arquitectura sólida** y a **comprender a fondo los requerimientos del sistema**. En esta fase, el modelo de análisis se utiliza para documentar y representar los artefactos obtenidos del proceso de análisis, colaborando al mantenimiento y la consistencia durante el ciclo de vida del software, que agrega como ventajas:

- ✓ **Identificación temprana de problemas:** Un modelo de análisis sólido permite identificar y corregir errores, inconsistencias o malas prácticas en una etapa temprana del proceso de desarrollo.
- ✓ **Reducción de costos:** Al corregir los problemas en una etapa temprana, se evita el gasto adicional asociado con la reestructuración y modificación del código ya implementado.
- ✓ **Evitar problemas de escalabilidad:** Las inconsistencias o malas prácticas no corregidas durante la etapa de análisis pueden conducir a problemas de escalabilidad a medida que el sistema crece en complejidad y tamaño.

Explique los tres tipos de clases que se utilizan para modelar el análisis, según el PUD y ejemplifíquelas.

Clases de análisis son aquellas que se centran en el **tratamiento de requisitos funcionales**, lo cual es más evidente en el contexto del dominio del problema. Son clases que **tienden a ser más conceptuales**, ya que su comportamiento se define mediante **responsabilidades** en un nivel más alto y menos formal. Estas clases también **definen atributos** a un nivel alto, siendo conceptuales y reconocibles en el dominio del problema

Clase de interfaz: Modela las **interacciones entre el sistema y los actores**. Esta interacción implica recibir (y presentar) información y peticiones de (y hacia) los usuarios y sistemas externos.

Clase de entidad: Modela la **información y el comportamiento asociado a algún fenómeno o concepto que posee vida larga** y que, a menudo, es **persistente**. Por lo general, derivan de clases de entidad del negocio (o del dominio del problema), reflejan información beneficiosa para los desarrolladores al diseñar e implementar dicha funcionalidad.

La **diferencia de clase de entidad respecto a clases de entidad del negocio** (del dominio), es que las clases del dominio describen el contexto del sistema y por lo tanto pueden incluir información que el sistema no maneja en absoluto.

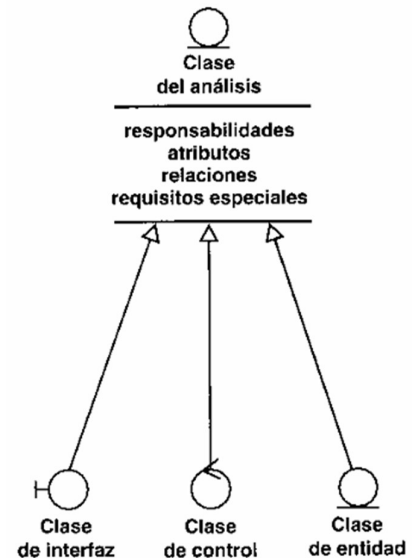
Clase de control: Representan coordinación, secuencia, transacciones y control de otros objetos, y se usan para encapsular el control de un CU en concreto. También representan cálculos y derivaciones complejas (lógica de negocio) que no pueden asociarse a ninguna clase de entidad concreta.

- Modela los aspectos dinámicos del sistema, manejan y coordinan las acciones y los flujos de control principales del CU, delegando trabajo a otros objetos.
- Son el nexo entre las clases de entidad e interfaz.

Describe las responsabilidades de los trabajadores involucrados en el Flujo de Trabajo de Análisis y vincúlelos con los artefactos que producen.

En el workflow de análisis, podemos encontrar 3 trabajadores:

Arquitecto: Es responsable de la integridad del **modelo de análisis**, garantizando que este sea correcto, consistente y legible como un todo. El modelo es correcto siempre y cuando cumpla con la funcionalidad descrita en el modelo de CU, y solo esa.



- ✓ También es responsable de la **arquitectura**, es decir, de la existencia de las partes significativas de esta.

Ingeniero de CU: Responsable de mantener la integridad de una o más **realizaciones de CU**, garantizando que cumplan los requerimientos que recaen sobre ellas. No es responsable de las clases de análisis ni de las relaciones que se usan en la realización del caso de uso.

Ingeniero de Componentes: Responsable de definir y mantener atributos, responsabilidades, relaciones y requerimientos especiales de una o más **clases de análisis**, garantizando que cumplen con los requerimientos que se esperan de ella.

- ✓ También es responsable de la integridad de los **paquetes del análisis**, garantizando que su contenido sea correcto.

Explique cuáles son los artefactos de salida del Workflow de Análisis y describa los diagramas de UML 2.0 que se utilizan para construirlos.

Los artefactos de salida del workflow de análisis son:

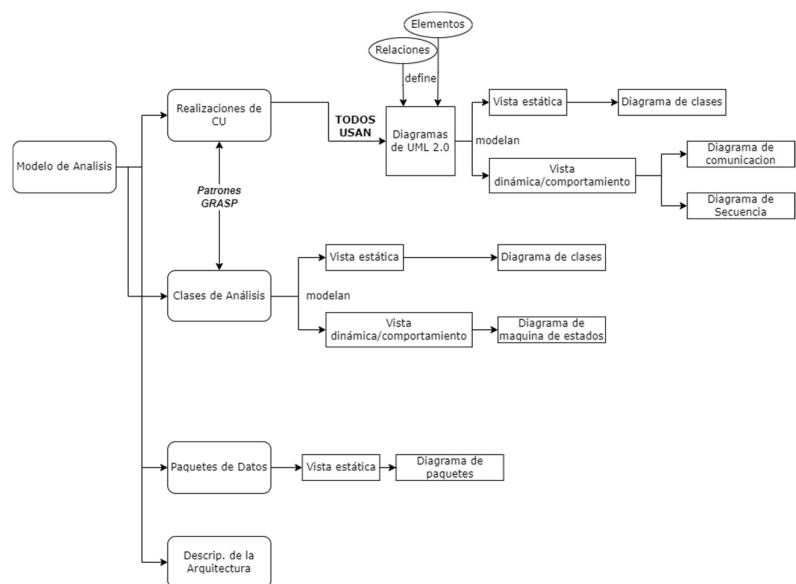
Modelo de análisis: Modelo de objetos conceptual, que nos ayuda a refinar y estructurar los requerimientos, nos permite razonar sobre aspectos internos del sistema, y nos proporciona una estructura centrada en el mantenimiento, en aspectos tales como la flexibilidad ante cambios y la reutilización.

Clases de análisis: Abstracción que modela un elemento específico del dominio del problema.

- Se centra en el tratamiento de requerimientos funcionales y pospone los no funcionales.
- Define atributos en un alto nivel.
- Participa en relaciones, más conceptuales que las de diseño e implementación.
- Su comportamiento se define mediante responsabilidades en un alto nivel.

Pueden ser de 3 tipos distintos, los cuales están estandarizados en UML y ayudan a los desarrolladores a distinguirlas:

1. **Clases de interfaz:** Modela las interacciones entre el sistema y sus actores. Estas consisten, a menudo, en presentar (y recibir) información (y peticiones) desde (y hacia) los usuarios y sistemas externos.
 2. **Clases de entidad:** Modela la información y el comportamiento asociado a algún fenómeno o concepto, que poseen vida larga y, a menudo, son persistentes.
- Por lo general, se derivan de clases de entidad del negocio (o del dominio del problema).
 - 3. **Clases de control:** Representan coordinación, secuencia, transacciones y control de otros objetos, y se usan para encapsular el control de un CU en concreto.
 - También representan derivaciones y cálculos complejos que no pueden asociarse a ninguna clase de entidad concreta. Modela los aspectos dinámicos del sistema. Son el nexo entre las clases de interfaz y entidad.



Realizaciones de CU de análisis: Representan una colaboración dentro del modelo de análisis, que describe como se lleva a cabo y ejecuta un caso de uso determinado, en término de las clases de análisis participantes y los objetos de estas que interactúan.

- Poseen una descripción textual del flujo de sucesos, un diagrama de clases que muestra las clases de análisis participantes, y un diagrama de interacción que muestra la realización de un escenario particular del caso de uso.

Paquetes del análisis: Proporcionan un medio para organizar los artefactos del modelo de análisis en piezas manejables. Estos pueden contener clases de análisis, realizaciones de CU de análisis y otros paquetes (recursivamente).

- Los paquetes deben ser altamente cohesivos (su contenido debe estar fuertemente integrado) y débilmente acoplados (deben depender mínimamente unos de otros).

Descripción de la arquitectura (Vista del modelo de análisis): Contiene una vista de la arquitectura del modelo de análisis, que muestra sus artefactos más significativos para la arquitectura. Estos son:

- La descomposición del modelo de análisis en paquetes de análisis y sus dependencias.
- Clases de análisis fundamentales.
- Realizaciones de CU de análisis que describan una funcionalidad importante y crítica.



Trabajadores y artefactos implicados en el análisis.

Explique por qué es necesario considerar el análisis como una etapa diferente a la captura de requisitos o del diseño y la implementación.

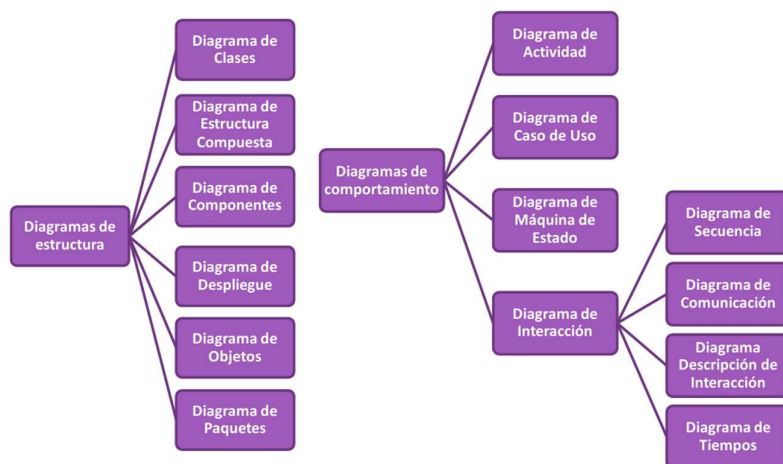
Se dice que **el análisis no es diseño ni implementación**, debido a que estas 2 etapas son mucho más que el análisis. Dichas etapas, se preocupan de dar forma al sistema, de manera que de vida a los requerimientos que incorpora, considerando atender los requerimientos de calidad (RNF), los cuales definirían los límites más tangibles del sistema, como su performance, seguridad, compatibilidad, etc.

Antes de comenzar a diseñar e implementar, es necesario contar con una **comprensión precisa y detallada de los requerimientos funcionales, la cual se consigue en la etapa de análisis.**

Por ello, se dice que con el análisis se consigue una **separación de intereses** que prepara y simplifica las actividades subsiguientes, delimitando los temas que deben resolverse y las decisiones a tomarse en dichas actividades.

UML 2.0:

¿Cuáles son los diagramas que UML 2.0? propone para crear modelos durante el Workflow de Análisis? Explique el propósito de cada uno, sus características y los conceptos que modela.



Los diagramas que UML 2.0 propone para crear modelos, durante el workflow de análisis, son:

- Diagrama de clases:** Muestran un conjunto de clases, interfaces y colaboraciones, así como sus relaciones. Modelan la parte estática del sistema.
- Diagrama de paquetes:** Muestra la descomposición del modelo de análisis en unidades organizativas, y sus dependencias.
- Diagrama de estados:** Muestra una máquina de estados, con sus respectivos estados, transiciones, eventos y actividades.
- Diagrama de comunicación:** Modela un conjunto de objetos, incluyendo los mensajes que intercambian entre sí.
Modelan la parte dinámica del sistema. Resaltan la organización estructural de dichos objetos.
- Diagrama de secuencia:** Ídem al de comunicación, con la diferencia que este resalta la ordenación temporal de los mensajes.
- Diagrama de timing:** Fusiona los diagramas de secuencia y de estado, y muestra los estados de un objeto a lo largo del tiempo, como así también los mensajes que afectan a este.
- Diagrama de descripción de la interacción:** Fusiona los diagramas de secuencia y de actividad, para permitir que los fragmentos de interacción sean fácilmente combinados con los puntos de decisión. Modelan la parte dinámica del sistema.

Mencione y describa los diagramas de UML 2.0 que se clasifican como diagramas de estructura.

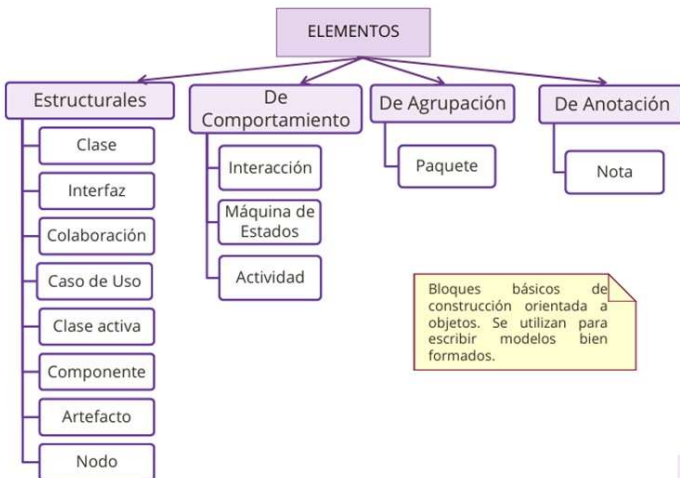
En UML 2.0, existen **13 diagramas**, **6** de los cuales se denominan **diagramas de estructura**. Estos son:

- Diagrama de clases:** Muestra un conjunto de clases, interfaces y colaboraciones, así como sus relaciones.
- Diagrama de objetos:** Muestra un conjunto de objetos, así como sus relaciones.
- Diagrama de paquetes:** Muestra la descomposición de un paquete en piezas manejables y sus dependencias.
- Diagrama de estructura compuesta:** Provee un medio para ordenar una estructura de elementos, y focalizar en detalles, construcción y relaciones internas.
- Diagrama de componentes:** Modela estructuras de alto nivel, construidas desde una o más clases, y provee una interfaz bien definida.

6. **Diagrama de despliegue:** Muestra la configuración de nodos de procesamiento en tiempo de ejecución, y los artefactos que residen en ellos.

Especifique como está compuesto el **modelo conceptual** de UML 2.0, y describa en términos generales los **elementos** que lo componen.

El modelo conceptual de UML 2.0 está compuesto por 3 elementos principales:



1) Bloques básicos de construcción.

a) Elementos.

✓ **Estructurales:** Partes **estáticas** de un modelo. Pueden ser:

- **Clase.** Abstracción de objetos
- **CU.** Interacción funcional
- **Interfaz.** Punto de interacción
- **Colaboración.** Trabajo conjunto entre objetos
- **Clase activa.** Clase con comportamiento
- **Componente.** Bloque de construcción
- **Artefacto.** Producto del desarrollo

- **Nodo.** Punto de conexión

✓ **De comportamiento:** Partes **dinámicas** de un modelo. Pueden ser:

- **Interacción.** Intercambio dinámico de mensajes
- **Máquina de estados.** Representación del comportamiento
- **Actividad.** Proceso o flujo

✓ **De agrupación:** Partes **organizativas** de un modelo. Pueden ser:

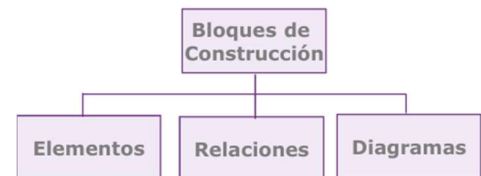
- **Paquetes.** Organización de elementos relacionados

✓ **De anotación:** Partes **explicativas** de un modelo. Pueden ser:

- **Notas.** Información adicional

b) Relaciones.

- ✓ **Asociación.** Representa una relación entre dos clases.
- ✓ **Agregación.** Relación "todo-parte" entre clases, donde una clase contiene instancias de otra.
- ✓ **Generalización.** Herencia de clases
- ✓ **Realización.** Implementación de interfaces



c) Diagramas.

- ✓ De **estructura** (6).
- ✓ De **comportamiento** (7).

2) **Reglas** que dictan como combinar estos bloques. Estas son reglas sintácticas y semánticas para:

- a) Nombres.
- b) Alcance.
- c) Visibilidad.
- d) Integridad.

e) Ejecución.

3) **Mecanismos comunes** que se aplican a través de UML. Estos son:

- a) Especificaciones.
- b) Adornos.
- c) Divisiones Comunes.
- d) Mecanismos de extensibilidad:
 - ✓ Estereotipos.
 - ✓ Valores etiquetados.
 - ✓ Restricciones.



¿Qué modelan los **diagramas de interacción**?

Realice una clasificación de cada uno de ellos.

Los diagramas de interacción modelan un conjunto de

objetos que se comunican intercambiando mensajes entre sí. Existen **4 tipos**:

- ✓ **Diagrama de secuencia**: Resalta la ordenación temporal de los mensajes.
- ✓ **Diagrama de comunicación**: Resalta la organización estructural de los objetos que se comunican.
- ✓ **Diagrama de timing**: Combina los diagramas de secuencia y de estados, para proveer una vista del estado de un objeto a través del tiempo, y los mensajes que lo modifican.
- ✓ **Diagrama de descripción de la interacción**: Fusiona los diagramas de secuencia y de actividad, para permitir que los fragmentos de interacción sean fácilmente combinados con los puntos y flujos de decisión.

Explique la diferencia entre los conceptos: Modelo, Vista y Diagrama. Ejemplifique.

Modelo: Es la representación simplificada de un sistema, elaborada para predecir, controlar y comprender el funcionamiento del sistema. Por ejemplo, se tiene el modelo de requerimientos para comprender y estudiar el dominio del problema.

Ejemplo: En el contexto del desarrollo de software, un modelo puede ser una representación conceptual de los requisitos, la arquitectura o el diseño de un sistema. Por ejemplo, un modelo de casos de uso representa las interacciones entre un sistema y sus usuarios, pero no describe detalles de implementación.

Vista: Es una perspectiva del sistema. Las vistas son 5: casos de uso; diseño; interacción; implementación y despliegue; y se ilustran a través de diagramas. Un ejemplo es una vista de diseño, que se ilustra estáticamente mediante un diagrama de clases, para comprender las clases y relaciones que forman el vocabulario del problema y la solución.

Diagrama: Es la representación de una perspectiva del sistema que se **ilustra la** mayoría de las veces como un grafo conexo de nodos y arcos. Permite visualizar partes del sistema e ilustrar como estas funcionan. Un ejemplo de diagrama es un diagrama de clases. En el cual se ilustran las clases participantes con sus **atributos, métodos** y relaciones.

¿Cuáles son los diagramas de UML 2.0 que **modelan objetos** (instancias de clases)?
 Describa cada uno de ellos y mencione en que workflows del Proceso Unificado se los puede utilizar.

Los diagramas de UML 2.0 usados para modelar objetos son:

- **Diagrama de objetos:** Muestran un conjunto de objetos, así también como sus relaciones.
- **Diagrama de interacción:** Modelan un conjunto de objetos y los mensajes que se intercambian entre sí:
- **Diagrama de secuencia:** Enfatiza en la ordenación temporal de los mensajes. Resalta el orden de los mensajes en función del tiempo. Muestra un conjunto de objetos y los mensajes enviados y recibidos por esos objetos.
- **Diagrama de comunicación:** Resalta la organización estructural de los objetos que se comunican. Provee una vista de la colaboración de objetos en un ambiente de tiempo real, distribuyendo funcionalidad en las clases, analizando los aspectos comportamentales de un sistema.
- **Diagrama de timing:** Combina los diagramas de secuencia y de estados, y muestra los estados por los que pasa un objeto a través del tiempo, así también como los mensajes que lo modifican. También muestra la interacción entre eventos dependientes del tiempo y las restricciones de tiempo y duración que los gobiernan.
- **Diagrama de descripción de la interacción:** Fusiona los diagramas de secuencia y de actividad, para permitir que los fragmentos de interacción sean fácilmente combinados con los nodos y flujos de decisión.

¿Cuáles son los diagramas que UML 2.0 propone para **modelar la estructura del software**? Considere los que podemos aplicar en el Flujo de Trabajo de análisis: explique el propósito de cada uno y en qué actividad del F.T. se utilizan.

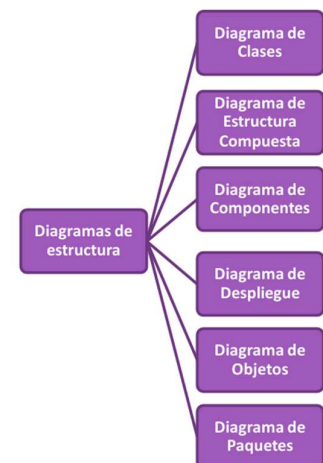
En el Flujo de Trabajo (FT) de análisis se suelen utilizar principalmente tres tipos de diagramas: El diagrama de clases, de objetos y de paquetes.

DIAGRAMA DE CLASE.

- El diagrama de Clase muestra los bloques de construcción de cualquier sistema orientado a objetos.
- Los diagramas de clases describen la vista estática del modelo o parte del modelo, describiendo que atributos y comportamientos tienen en lugar de detallar los métodos para realizar operaciones.
- Los diagramas de Clase son más útiles para ilustrar relaciones entre clases e interfaces. Las generalizaciones, agregaciones, y asociaciones son todas valiosas al reflejar herencias, composición o uso, y conexiones respectivamente.
- Actividad del FT: Se utiliza principalmente en la actividad de modelado conceptual y análisis de requisitos. Ayuda a identificar las entidades clave del sistema, sus relaciones y comportamientos básicos.

DIAGRAMA DE OBJETOS.

- Un diagrama de objetos puede ser considerado un caso especial de un diagrama de clase.



- Los diagramas de objetos utilizan un subconjunto de los elementos de un diagrama de clases con el fin de enfatizar la relación entre las instancias de clases en algún momento en el tiempo. Son útiles en la comprensión de diagramas de clases.
- No muestran nada diferentes en arquitectura a los diagramas de clases, sino que reflejan la multiplicidad y roles.
- Actividad del FT: Se utiliza para explorar y entender cómo interactúan los objetos del sistema en escenarios específicos. Puede ayudar a identificar patrones de comportamiento y aclarar aspectos del diseño.

DIAGRAMA DE PAQUETES.

- Los diagramas de paquetes se usan para reflejar la organización de paquetes y sus elementos.
- Los usos más comunes para los diagramas de paquete son para organizar diagramas de casos de uso y diagramas de clase, a pesar de que el uso de los diagramas de paquete no es limitado a estos elementos UML.
- Actividad del FT: Se utiliza en la actividad de modelado arquitectónico y diseño del sistema. Ayuda a organizar y visualizar la estructura del sistema en términos de módulos o componentes, lo que facilita la comprensión y comunicación entre los miembros del equipo.

Patrones GRASP:

Mencione los patrones GRASP utilizados en el Análisis, describa cuál es su utilidad y desarrolle un ejemplo PROPIO en el que aplique los patrones CREADOR y BAJO ACOPLAMIENTO.

Los patrones GRASP usados durante el análisis son:

- Experto en información:** Asigna responsabilidades al experto en información, es decir, a la clase que cuenta con la información necesaria para realizar la responsabilidad.
- Creador:** Asigna a la clase A la responsabilidad de crear una instancia de la clase B, si se cumple alguno de estos casos:
 - A está formado por B.
 - A contiene a B.
 - A está compuesto por B.
 - A tiene información para crear a B.
 - A utiliza a B.
- Bajo Acoplamiento:** Asigna responsabilidades a un elemento manteniendo un bajo nivel de acoplamiento.
 - El acoplamiento es una medida de la fuerza con que un elemento está conectado, tiene conocimientos o confía en otros elementos.
 - Un elemento débilmente acoplado, es aquel que no depende demasiado de otros elementos.
- Alta Cohesión:** Asigna responsabilidades a un elemento, manteniendo un alto nivel de cohesión.
 - La cohesión mide que tan relacionadas están las responsabilidades de un elemento.
 - Un elemento con alta cohesión posee responsabilidades altamente relacionadas, y no hace una gran cantidad de trabajo.
- Controlador:** Asigna responsabilidades de recibir o manejar un evento del sistema, a una clase que puede representar:
 - El sistema global.

- Un escenario de CU donde tiene lugar el evento del sistema.
- Sirve como intermediario entre la interfaz y el algoritmo que la implementa, recibiendo datos del usuario y enviándolos a las distintas clases, dependiendo del método llamado.
- Básicamente, delega en otros objetos el trabajo que necesita hacer, coordina y controla la actividad.

¿Para qué se aplican los Patrones GRASP? ¿En qué actividad del F.T. de Análisis se hace uso de los mismos?

- Un **patrón es una descripción del problema y su solución**, que posee un nombre, se puede **aplicar en diferentes contextos** e incluye consejos de cómo aplicarlos.
- Pretenden codificar **conocimientos**, estilos y principios existentes que **ya han sido probados que son válidos**.
- Los patrones GRASP **describen los principios fundamentales del diseño de objetos**, y se aplican para la asignación de responsabilidades.

En el Framework de Trabajo (FT) de Análisis, los Patrones GRASP se utilizan especialmente en la actividad de definición y análisis de requerimientos. Durante esta actividad, se aplican los Patrones GRASP para comprender y modelar las responsabilidades de las diferentes clases y objetos del sistema, lo que ayuda a establecer una arquitectura sólida y coherente, promoviendo buenas prácticas y claridad del sistema en sí.

Completar el siguiente cuadro para que el patrón se corresponda con la afirmación:

Característica del patrón	Patrón GRASP
Asigna la responsabilidad a la clase que cuenta con la información necesaria para realizar la responsabilidad.	Experto
Asigna la responsabilidad a un elemento, de recibir o manejar un evento del sistema.	Controlador
Asigna la responsabilidad al elemento A, de crear una instancia del elemento B.	Creador
Asigna la responsabilidad a un elemento, de tal manera que sus responsabilidades estén altamente relacionadas entre sí.	Alta Cohesión
Asigna la responsabilidad a un elemento, de modo tal que este no dependa demasiado de otros elementos.	Bajo Acoplamiento

Para el patrón **Experto** plantee un ejemplo PROPIO concreto de:

a) Interacción que respeta la aplicación del patrón Experto:

Supongamos que tenemos un sistema de gestión de una biblioteca. Para calcular la multa por el retraso en la devolución de un libro, creamos una clase llamada GestorMulta que tiene la responsabilidad de calcular la multa basada en la fecha de devolución y la fecha límite. Esta clase es un experto en la lógica de cálculo de multas porque tiene toda la información necesaria para realizar este cálculo de manera precisa.

b) Interacción que no respeta la aplicación del patrón Experto:

En el mismo sistema de gestión de la biblioteca, podríamos tener una clase llamada Libro que tiene la responsabilidad de calcular la multa por el retraso en la devolución. Sin embargo, esto violaría el principio de experto, ya que la clase Libro debería estar más centrada en representar la información relacionada con el libro (título, autor, ISBN, etc.) y no debería tener la responsabilidad de calcular multas. Esto rompería la cohesión de la clase Libro y crearía acoplamiento innecesario entre diferentes aspectos del sistema.

¿Cuál es la utilidad de los patrones GRASP? Explique su propósito y desarrolle un ejemplo PROPIO en el que use el patrón CONTROLADOR y un ejemplo en el que no lo use.

Los patrones GRASP describen los principios fundamentales del diseño de objetos, y la asignación de responsabilidades, expresados como patrones.

El patrón CONTROLADOR se utiliza para asignar la responsabilidad del manejo de un mensaje de los eventos del sistema a una clase controladora. **Su propósito es garantizar que los procesos del dominio sean manejados por la capa del dominio y no por la de interfaz**, lo que facilita la modularidad y el mantenimiento del sistema.

Ejemplo con el patrón CONTROLADOR:

Imagina un sistema de gestión de biblioteca. Cuando un usuario realiza una solicitud de préstamo de un libro a través de la interfaz de usuario, el controlador de préstamos se encarga de procesar esta solicitud. Este controlador valida la disponibilidad del libro, verifica el estado del usuario y actualiza la base de datos con la información del préstamo. Al separar esta lógica de procesamiento de la interfaz de usuario, el sistema se vuelve más modular y fácil de mantener.

Ejemplo sin utilizar el patrón CONTROLADOR:

En el mismo sistema de gestión de biblioteca, si no se aplica el patrón CONTROLADOR, es posible que la lógica de procesamiento de la solicitud de préstamo se implemente directamente en la interfaz de usuario. Esto podría resultar en una interfaz de usuario más compleja y difícil de mantener, ya que la lógica de negocio estaría entrelazada con la lógica de presentación. Además, cualquier cambio en la forma en que se manejan las solicitudes de préstamo requeriría modificaciones en la interfaz de usuario, lo que aumentaría la fragilidad y la complejidad del sistema.

Resumen de clasificación de PATRONES GRASP.

Patrón	Problema	Solución	Explicación	Beneficios
Controlador	¿Quién debería encargarse de atender un evento del sistema?	Asignar la responsabilidad del manejo de un mensaje de los eventos de un sistema, a una clase controladora.	Un controlador de caso de uso es una buena alternativa cuando hay muchos eventos del sistema entre varios procesos.	Garantiza que los procesos del dominio sean manejados por la capa del dominio y no por la de interfaz. Reflexiona sobre el estado de un caso de uso.
Experto	¿Cuál es el principio fundamental en virtud del cual se asignan las responsabilidades?	Asignar una responsabilidad al experto en información.	Los objetos hacen cosas relacionadas con la información que poseen. Puede haber expertos parciales.	Bajo acoplamiento. El comportamiento se distribuye entre las clases que cuentan con la información requerida.

Patrón	Problema	Solución	Explicación	Beneficios
Creador	¿Quién debería ser el responsable de crear una nueva instancia de alguna clase?	Asignar a la clase B la responsabilidad de crear una instancia de la clase A.	Encontrar un creador que debemos conectar con el objeto producido, esto soporta el bajo acoplamiento.	Ayuda al bajo acoplamiento, supone menos dependencias. La clase creada tiende a ser visible al creador.
Bajo Acoplamiento	¿Cómo dar soporte a una dependencia escasa y a un aumento de reutilización?	Asignar una responsabilidad para mantener bajo el acoplamiento.	Clases más independientes, que reducen el impacto de los cambios, y también más reutilizables.	No se afectan componentes por cambios de otros componentes. Fáciles de entender y reutilizar.
Alta Cohesión	Las clases con baja cohesión a menudo representan un grado de abstracción alto...	Asignar una responsabilidad de modo que la cohesión siga siendo alta.	Una clase de alta cohesión tiene un número relativamente pequeño de operaciones con funcionalidad relacionada.	Mejora la claridad y la facilidad de comprensión. Simplifica la evolución. A menudo genera bajo acoplamiento. Mayor capacidad de reutilización.